

Software Requirements Specification (SRS)

Blockchain Voting System — Major Project

Document Version:	1.0 Final Draft
Prepared By:	TYBSc Computer Science / AI-ML Project Team
Institution:	Thakur Ramnarayan College of Arts & Commerce (TRCAC)
Date:	August 2026
Scope:	Full-Stack Decoupled Architecture (FastAPI + React 19)
Status:	Production-Ready Release

Table of Contents

- 1. Introduction (Purpose, Scope, Definitions, References, Overview)
- 2. Overall Description (Product Perspective, Functions, User Classes, Constraints)
- 3. User Roles & Characteristics (Admin, Voter, Observer)
- 4. Functional Requirements (Auth, Election, Vote, Blockchain, Observer, AI Fraud)
- 5. Non-Functional Requirements (Performance, Security, Reliability, Usability, Scalability)
- 6. System Constraints & Assumptions
- 7. External Interface Requirements (UI, Hardware, Software, Communication)
- 8. Data Requirements (Data Entities, Retention, Integrity)
- 9. UML Diagrams & Architecture Diagrams Set (Use Case, Class, Sequence, ER)
- 10. Architecture Design Document (ADD) & Key Decisions Matrix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the complete functional, non-functional, and architectural requirements for the **Blockchain Voting System**. This document serves as the primary engineering contract for developer teams, academic evaluators, security auditors, and system administrators.

1.2 Scope

The Blockchain Voting System is a modern web-based platform designed to conduct tamper-evident digital elections. It features a custom Python cryptographic blockchain engine, role-based access control, AES-256 GCM vote payload encryption, ECDSA SECP256R1 digital signatures, real-time WebSockets analytics, an AI Fraud Radar module, and downloadable QR code receipts.

Role / User Class	Key Capabilities & Module Privileges
Admin	Full election creation, candidate studio, bulk voter CSV import, system metrics, audit logs, blockchain health.
Voter	View active elections, cast 1-click encrypted vote, download QR receipts, verify vote hash on ledger.
Observer	Read-only transparency desk, live Recharts vote tallies, blockchain block explorer, AI fraud radar.

1.3 Definitions & Acronyms

Term	Definition
SRS	Software Requirements Specification
PoW	Proof-of-Work mining consensus algorithm
ECDSA	Elliptic Curve Digital Signature Algorithm (SECP256R1)
AES-256 GCM	Advanced Encryption Standard with Galois/Counter Mode
JWT	JSON Web Token for stateless authorization
RBAC	Role-Based Access Control
Merkle Tree	Cryptographic hash tree aggregating block transactions

1.4 Overview

The system is structured into three decoupled tiers: a React 19 + Vite frontend Single Page Application (SPA), a FastAPI (Python 3.12) REST API backend connected to SQLite / PostgreSQL, and an in-memory custom SHA-256 Blockchain ledger engine.

2. Overall Description

2.1 Product Perspective

The Blockchain Voting System operates as a standalone full-stack web application replacing paper-based ballot casting. It enforces voter anonymity while providing verifiable audit trails via public block hashes, Merkle roots, and QR receipts.

2.2 Product Functions (High-Level)

- 1. User Authentication & JWT Authorization — Secure role logins (Admin, Voter, Observer) with bcrypt password hashing.
- 2. Election Management — Admin creates, edits, activates, schedules, and closes elections.
- 3. Candidate Management — Admin creates candidate profiles with manifesto details and avatar URLs.
- 4. Bulk Voter Import — Admin uploads CSV files to enroll voters in bulk.
- 5. Vote Casting — Single vote per user per election enforced via database constraints and voter cryptographic hash.
- 6. AES-256 Vote Payload Encryption — Candidate selections encrypted prior to block inclusion.
- 7. ECDSA Digital Signature Signing — Transactions signed using single-use voter ECDSA key pairs.
- 8. Proof-of-Work Block Mining — Mined blocks aggregate queued transactions with SHA-256 difficulty targets.
- 9. QR Code Receipt Generation — Downloadable receipts containing receipt hash, block index, and transaction hash.
- 10. Blockchain Explorer — Public block browser, transaction search, and 1-click ledger tamper auditor.
- 11. AI Fraud Radar — Analyzes vote velocity bursts (<2s), duplicate IP concentration, and flags threat scores (0-100%).
- 12. Real-Time WebSockets — Live broadcast of vote tallies, block mining events, and active standings.

2.3 Operating Environment

Component	Specification
Client Browser	Modern web browsers (Chrome, Firefox, Edge, Safari)
Frontend Runtime	Node.js v20+, Vite 5.x, React 19, Tailwind CSS 3.4
Backend Server	Python 3.12+, FastAPI 0.111+, Uvicorn ASGI Server
Database Engine	SQLite (dev mode) / PostgreSQL (production mode)
Deployment Containers	Docker, Docker Compose, GitHub Actions CI/CD

3. User Roles & Characteristics

User Role	Technical Level	Key Responsibilities & System Privileges
Admin	Moderate to High	Full administrative control. Creates elections, manages candidates, uploads CSV voter lists, views system metrics, monitors blockchain health, reviews audit logs.
Voter	Basic to Moderate	Registered eligible voter. Views active elections, casts single encrypted vote, generates QR code receipt, verifies receipt hash on ledger.
Observer	Basic to High	Public auditor. Read-only access to live election standings, vote percentage charts, block explorer, tamper verification, and AI fraud radar.

4. Functional Requirements

FR-AUTH-01: User Registration & Login

System allows registration and login using email/username and password. Passwords are hashed using bcrypt with salt rounds. Returns JWT access and refresh tokens.

FR-AUTH-02: Role-Based Route Protection

Frontend router and backend endpoints enforce role checking (admin, voter, observer). Unauthorized requests return HTTP 401/403.

FR-ELEC-01: Election Lifecycle Management

Admin can create elections with title, description, start/end time. Elections transition through statuses: draft -> scheduled -> active -> closed.

FR-ELEC-02: Candidate Studio

Admin can register candidates for an election with name, party affiliation, manifesto description, and avatar URL.

FR-VOTE-01: 1-Click Encrypted Voting

Voters can select an active election candidate and submit a vote. Vote payload is encrypted using AES-256 GCM, signed with ECDSA SECP256R1, and mined into a block.

FR-VOTE-02: Anti-Double Voting Enforcement

System checks if a voter has already cast a vote in the target election. Duplicate attempts are blocked and logged in audit trails.

FR-VOTE-03: Downloadable QR Receipt

Upon vote confirmation, system generates a QR receipt containing receipt hash, block index, transaction hash, and voter anonymous hash.

FR-CHAIN-01: Proof-of-Work Block Mining

Transactions are packaged into blocks. Miner computes SHA-256 block hash meeting difficulty target (e.g. leading zeros).

FR-CHAIN-02: Merkle Tree Integrity

Block header includes binary Merkle Root hash computed from contained transaction hashes. Recomputed during ledger audits.

FR-CHAIN-03: Blockchain Explorer

Public endpoint allowing users to view mined blocks, search by block index or transaction hash, and trigger 1-click cryptographic chain verification.

FR-AI-01: AI Fraud Radar & Velocity Analysis

AI service monitors vote submission velocity (<2s bursts), duplicate IP concentration, and double-vote attempts to output a 0-100% Fraud Risk Index.

5. Non-Functional Requirements

Category	Requirement Specification
Performance	API response time < 500ms for REST endpoints; block mining time < 2s under PoW difficulty 2.
Security	AES-256 GCM vote payload encryption, ECDSA digital signatures, bcrypt password hashing, JWT authorization.
Reliability	99.9% uptime. Immutable ledger guarantees 0 vote loss or unauthorized block modifications.
Usability	Responsive glassmorphism UI built with Tailwind CSS, Framer Motion animations, Dark/Light mode toggle.
Scalability	Stateless JWT architecture allows horizontal scaling; database compatible with SQLite and PostgreSQL.
Maintainability	Modular clean architecture separating API routes, database models, Pydantic schemas, and Blockchain services.

6. System Constraints

1. Blockchain difficulty is set to 2 zero-prefix target hashes for instant MVP demonstration without excessive CPU usage.
2. Mock verification is used for MVP voter enrollment without requiring paid SMS/Email OTP gateways.
3. Custom Python Blockchain operates as an in-memory/DB-persisted engine, designed so Ethereum/Solidity can be integrated later without altering core APIs.
4. Single-vote policy is strictly enforced per user account per election.

7. Architecture & System Flow

7.1 Three-Tier Decoupled Model

Tier 1 (Presentation Layer): React 19 + Vite + Tailwind CSS Single Page Application.

Tier 2 (Application Layer): FastAPI REST API + WebSockets connection manager + AI Fraud Radar.

Tier 3 (Data & Ledger Layer): Custom SHA-256 Blockchain Engine + SQLAlchemy DB (SQLite / PostgreSQL).

7.2 Cryptographic Voting Workflow

- Step 1: Voter selects candidate in active election -> Frontend triggers POST /api/v1/votes/cast.
- Step 2: Backend verifies user hasn't voted & generates single-use ECDSA key pair and voter hash.
- Step 3: Vote choice encrypted using AES-256 GCM -> Digital signature generated using voter private key.
- Step 4: Transaction added to pool -> Proof-of-Work miner finds block nonce matching target difficulty.
- Step 5: Mined block appended to chain -> Database synced -> Real-time WebSocket event broadcast to dashboards.
- Step 6: QR Code Receipt generated containing receipt hash, block index, and transaction hash for verification.

8. Data Requirements & Schema

Table / Entity	Primary Key	Key Fields & Constraints
users	id	email (unique), username (unique), hashed_password, role (admin/voter/observer), is_active
elections	id	title, description, status (draft/scheduled/active/closed), start_time, end_time, created_by
candidates	id	election_id (FK), name, party, manifesto, avatar_url, vote_count
votes	id	user_id (FK), election_id (FK), candidate_id (FK), voter_hash, encrypted_vote, tx_hash (unique), block_index, receipt_hash (unique)
blocks	id	index (unique), timestamp, previous_hash, hash (unique), nonce, merkle_root, signature
transactions	id	tx_hash (unique), block_index, election_id, voter_hash, encrypted_vote, timestamp, signature
audit_logs	id	user_id, user_email, action, details, ip_address, timestamp

9. Key Design Decisions Matrix

#	Decision	Selected Choice	Rationale
1	Cryptographic Ledger	Custom Python Blockchain	Zero gas fees, lightweight, complete control over block structure & PoW difficulty.
2	Vote Secrecy	AES-256 GCM Encryption	Prevents plaintext vote choices from being visible in public block explorer.
3	Voter Authenticity	ECDSA SECP256R1 Signatures	Asymmetric key pair ensures transaction cannot be forged or tampered.
4	Real-Time Updates	FastAPI WebSockets	Instant push broadcasts of votes and block additions to dashboards without polling.
5	AI Threat Radar	Heuristic Velocity Engine	Detects vote bursts (<2s), duplicate IP clusters, and double-voting attempts.